Registro de Decisiones de Arquitectura (ADR)

Plataforma de Aprendizaje Gamificado

Versión 1 — OFICIAL

Fecha del documento: 13 de septiembre de 2026

ADR-0001: Elección de lenguaje/plataforma de backend

Fecha: 13 de septiembre de 2026

Estado: Aceptada

Contexto

El proyecto es una plataforma web con suscripción paga, desarrollada por un equipo de 1 persona (máximo 2). Adrián tiene experiencia previa en Java. Se necesita elegir un lenguaje/plataforma de backend que permita avanzar rápido, con buen soporte de seguridad y un ecosistema maduro.

Opciones consideradas

- Java con Spring Boot
- Kotlin (sobre JVM, interoperable con Spring Boot)
- Node.js con TypeScript

Decisión

Se elige Node.js como plataforma de backend.

Trade-offs

Aspecto	Análisis
A favor	Mayor velocidad de desarrollo para un equipo de 1-2 personas; menor configuración inicial que Spring Boot; permite compartir lenguaje entre frontend y backend (JavaScript/TypeScript); ecosistema npm muy amplio; alineado con lo que se recomienda en el entorno académico de Adrián.
En contra / riesgos	Adrián no tiene experiencia previa en el lenguaje (curva de aprendizaje); la seguridad (ej. autenticación, validación) debe componerse manualmente con librerías, a diferencia de Spring Security que la trae integrada; se debe ser disciplinado para mantener buenas convenciones sin un framework tan prescriptivo como Spring Boot.
Alternativa descartada	Java/Kotlin + Spring Boot habría aprovechado el conocimiento previo de Java y ofrece seguridad robusta de fábrica (Spring Security), pero con mayor complejidad de configuración inicial y una curva de productividad más lenta para un equipo tan pequeño.

ADR-0002: Arquitectura: monolito modular vs. microservicios

Fecha: 13 de septiembre de 2026

Estado: Aceptada

Contexto

El equipo de desarrollo es de 1 persona (máximo 2). Se necesita una arquitectura que sea mantenible, organizada, con fronteras claras entre funcionalidades, y que no imponga una carga operativa innecesaria para un equipo tan pequeño.

Opciones consideradas

- Arquitectura de microservicios
- Monolito tradicional (sin división interna clara)
- Monolito modular (un solo despliegue, dividido internamente en módulos con fronteras definidas)

Decisión

Se elige un monolito modular, con módulos como auth, suscripciones, contenido, desafíos, gamificación y administración, cada uno con fronteras claras y comunicación controlada entre ellos.

Trade-offs

Aspecto	Análisis
A favor	Un solo despliegue simplifica la operación para un equipo pequeño; las fronteras internas evitan que un cambio en un módulo rompa otros; facilita mantener el código organizado a medida que crece; deja la puerta abierta a separar un módulo en un servicio independiente en el futuro si fuera necesario.
En contra / riesgos	Si no se respetan las fronteras entre módulos con disciplina, el monolito puede degradar hacia un "monolito espagueti"; escalar un módulo específico de forma independiente (ej. solo gamificación) no es tan directo como en microservicios.
Alternativa descartada	Microservicios habría permitido escalar y desplegar cada módulo de forma independiente, pero introduce complejidad operativa (orquestación, comunicación entre servicios, múltiples despliegues) que no se justifica para un equipo de 1-2 personas en esta etapa.

ADR-0003: Estrategia de escalabilidad para soportar cientos de usuarios concurrentes

Fecha: 13 de septiembre de 2026

Estado: Aceptada

Contexto

La plataforma debe estar preparada, desde el inicio, para que cientos de estudiantes puedan usarla simultáneamente (navegar contenido, resolver desafíos, consultar el ranking), manteniendo la decisión de arquitectura de monolito modular (ADR-0002).

Opciones consideradas

- No planear escalabilidad de entrada y optimizar solo si aparecen problemas de rendimiento
- Migrar a microservicios para escalar cada módulo por separado
- Mantener el monolito modular, pero incorporar desde el diseño buenas prácticas de escalabilidad (pool de conexiones, backend sin estado, caché, índices, procesamiento asíncrono para tareas pesadas)

Decisión

Se mantiene el monolito modular, incorporando desde el diseño: pool de conexiones a MySQL, backend stateless (sin sesión en memoria del servidor) para permitir escalado horizontal si es necesario, uso de caché (ej. Redis) para datos de alta lectura como el ranking y el contenido del pensum, índices adecuados en las tablas de mayor consulta, y procesamiento asíncrono (colas) para tareas pesadas como el recálculo del ranking.

Trade-offs

Aspecto	Análisis
A favor	Permite atender cientos de usuarios concurrentes sin necesidad de rediseñar la arquitectura; mantiene la simplicidad operativa del monolito modular; deja el sistema preparado para escalar horizontalmente (varias instancias) si el crecimiento lo exige.
En contra / riesgos	Añade algo de complejidad desde el inicio (ej. configurar caché, colas) que un monolito simple no tendría; requiere disciplina para no introducir estado en memoria del servidor a medida que se agreguen funcionalidades.
Alternativa descartada	No planear escalabilidad desde el inicio es más simple a corto plazo, pero implica un riesgo alto de caída o lentitud de la plataforma justo cuando empiece a crecer la base de estudiantes, que es un momento crítico para la reputación de una plataforma de suscripción.